T01: Fundamentos de Docker (Manual de Supervivencia)

> **Módulo:** Despliegue de Aplicaciones Web (2º DAW) / Sistemas y Redes (2º DAM)
> **Ciclos Formativos:** DAW & DAM — IES La Mola
> **Proyecto:** Pizzería Bella Napoli
> **Metodología:** *Top-Down* — Destripando la infraestructura real de producción.

En este manual vamos a desmontar y entender la tecnología subyacente que hace funcionar nuestra pizzería en AWS, permitiéndote ser 100% autónomo para empaquetar, depurar y desplegar cualquier proyecto en el mundo profesional.

Índice del Manual

1. De la Máquina Virtual al Contenedor
2. Imágenes vs. Contenedores (La Receta y la Pizza)
3. Destripando el Proyecto: El Dockerfile del Backend
4. El Kit de Supervivencia: Ciclo de Vida y Terminal
5. Persistencia y Redes Aisladas
6. Docker Compose: El Director de Orquesta
7. Chuleta Rápida de Comandos (Cheat Sheet)
8. Guía Rápida de Troubleshooting (Resolución de Incidencias)

Capítulo 1: De la Máquina Virtual al Contenedor

El problema histórico: *"En mi máquina funciona"*

Imagina que terminas de programar una API en Node.js. En tu ordenador portátil funciona a la perfección. La subes a un servidor en la nube (nuestra instancia EC2 con Ubuntu) y de repente explota:

- En tu portátil tenías Node.js 20, pero el servidor tiene Node.js 16 por defecto.
- En tu portátil tenías una variable de entorno configurada y en el servidor no existe.
- El puerto 3000 ya lo está usando otro proceso del sistema operativo.

La solución clásica: Máquinas Virtuales (VM)

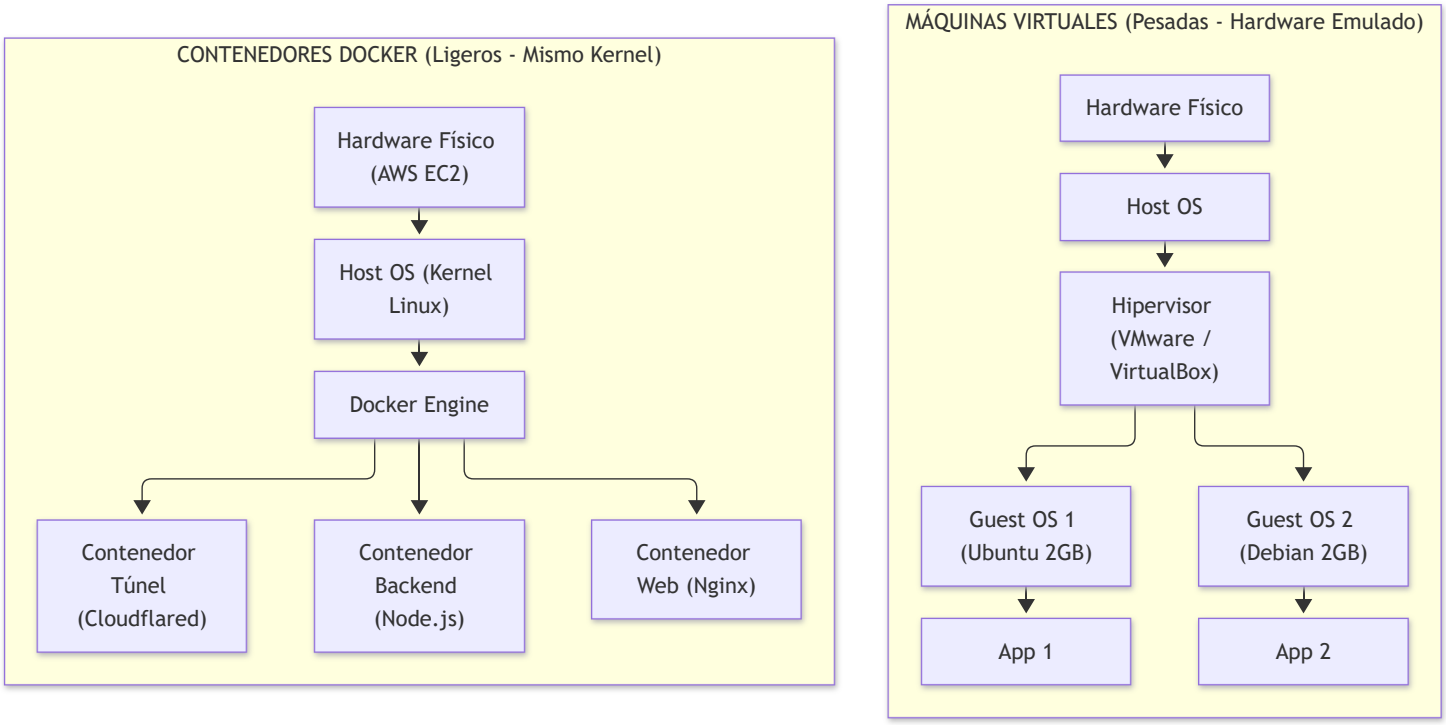
Para solucionar esto, históricamente se creaban Máquinas Virtuales (VirtualBox, VMware, KVM). Una VM emula un ordenador completo desde el hardware virtualizado hasta el Sistema Operativo huésped (*Guest OS*).

- **Desventajas:** Cada VM pesa varios Gigabytes, tarda minutos en arrancar y desperdicia muchísima memoria RAM y CPU duplicando sistemas operativos completos.

La solución moderna: Contenedores Docker

Docker no virtualiza hardware; **virtualiza el Sistema Operativo**.

Un contenedor empaqueta tu código fuente junto con sus dependencias exactas, pero **comparte el núcleo (Kernel de Linux) de la máquina anfitriona** (nuestra instancia EC2).



¿Por qué lo usamos en la Pizzería Bella Napoli?

En lugar de instalar Node.js, Nginx y librerías directamente sobre el sistema operativo de nuestra máquina en AWS, hemos metido cada pieza en su propia **"caja estanca"**. Si el backend sufre un desbordamiento de memoria o se apaga, Nginx y la base de datos continúan operando sin verse afectados.

Capítulo 2: Imágenes vs. Contenedores (La Receta y la Pizza)

Para entender Docker es fundamental dominar esta analogía:

- **La Imagen (La Receta / La Plantilla):**
  Es un archivo estático, inmutable y de solo lectura compuesto por **capas superpuestas** (*layers*). Contiene el sistema operativo base mínimo, las librerías instaladas y tu código fuente. No está "viva" ni consume memoria RAM.
- **El Contenedor (La Pizza Horneada / La Instancia):**
  Es la ejecución viva de una imagen en memoria. Añade una fina capa escribible en la parte superior. A partir de una sola imagen de backend podemos levantar 1, 3 o 50 contenedores idénticos funcionando a la vez en distintos puertos.

$$\text{Dockerfile (Código)} \xrightarrow{\text{docker build}} \text{Imagen (Receta estática en capas)} \xrightarrow{\text{docker run}} \text{Contenedor (Proceso en memoria)}$$

¿Dónde viven las imágenes y de dónde salen?

Una duda lógica al empezar con Docker es: *"Si abro la carpeta de mi proyecto en el explorador de archivos, no veo ningún archivo de imagen de 150 MB. ¿Dónde están? ¿Cómo han llegado hasta aquí?"*.

Para entender el ciclo completo, debemos distinguir dos entornos:

1. Docker Hub: El almacén central en la nube

Igual que cuando haces `npm install` tu ordenador acude a un registro público en Internet para descargarse librerías, Docker cuenta con su propio registro oficial mundial: **Docker Hub** (el "GitHub de los contenedores").

- Comunidades y empresas oficiales como Node.js, Nginx, PostgreSQL o Cloudflare publican allí imágenes prefabricadas y optimizadas.
- Cuando en tu `Dockerfile` escribes `FROM node:20-alpine`, Docker se conecta automáticamente a Docker Hub y se descarga esa imagen base oficial si no la tiene guardada en tu máquina.
- También podemos usar imágenes directamente sin necesidad de crear un Dockerfile: en nuestro docker-compose.app.yml, el túnel seguro usa `image: cloudflare/cloudflared:latest`, que Docker descarga lista para usar desde Docker Hub.

2. El Almacén Local de tu Máquina (`docker images`)

Una vez descargada de Internet o cocinada con `docker build`, la imagen **no se guarda como un archivo suelto (.exe o .zip) en tu carpeta de trabajo**.

- Docker almacena las imágenes en una base de datos interna gestionada por el propio motor de Docker en el disco duro de la máquina (en Linux Ubuntu, dentro de `/var/lib/docker/`).
- Para inspeccionar qué imágenes tienes almacenadas físicamente en tu servidor y cuánto espacio ocupan, ejecutas en la terminal:

  ```
  docker images
  ```

  Verás su nombre (*Repository*), su etiqueta (*Tag*, ej. `20-alpine`), su identificador único (*Image ID*) y su tamaño en disco.

3. ¿Por qué NO subimos las imágenes a GitHub?

En el repositorio de Git de la pizzería solo guardamos **la receta escrita en texto plano** (`Dockerfile` y archivos de Compose).

- Subir imágenes completas a Git sería una pésima práctica de ingeniería: una imagen binaria pesa entre 30 MB y 500 MB. Si 20 alumnos subieran sus imágenes, el repositorio pesaría gigabytes y colapsaría.
- Con el código fuente y el `Dockerfile` en Git (que ocupan apenas unos pocos Kilobytes), cualquier máquina del mundo puede descargarse el repositorio y cocinar exactamente la misma imagen idéntica en pocos segundos ejecutando `docker compose up --build`.

Capítulo 3: Destripando el Proyecto: El Dockerfile del Backend

Un `Dockerfile` es el fichero de instrucciones donde le explicamos a Docker cómo debe cocinar la imagen de nuestra aplicación.

A continuación analizamos el archivo real de nuestra pizzería (backend/Dockerfile):

```
# 1. Imagen base ultraligera
FROM node:20-alpine

# 2. Carpeta de trabajo dentro del contenedor
WORKDIR /app

# 3. Copiar manifiestos de dependencias
COPY package*.json ./

# 4. Instalar dependencias de producción
RUN npm install --omit=dev

# 5. Copiar el resto del código fuente del backend
COPY . .

# 6. Exponer el puerto de la API REST
EXPOSE 3000

# 7. Comando de arranque
CMD ["node", "src/server.js"]
```

Análisis Línea a Línea:

1. `FROM node:20-alpine`

- **¿Qué hace?** Establece la imagen base oficial sobre la que construiremos la nuestra.
- **El truco de ingeniería:** Usamos la etiqueta `alpine`. Alpine Linux es una distribución hiperligera de Linux de tan solo **~5 MB** de base. La imagen completa con el motor de Node.js pesa apenas **~140 MB**, frente a los casi **~1.000 MB** (1 GB) que ocupa la versión de Node sobre Debian/Ubuntu estándar. Esto acelera las descargas y reduce drásticamente las vulnerabilidades potenciales de seguridad (*superficie de ataque*).

2. `WORKDIR /app`

- **¿Qué hace?** Equivale a un comando `mkdir -p /app && cd /app` dentro del contenedor. Todo lo que ejecutemos a partir de esta línea ocurrirá dentro de esa carpeta.

3. `COPY package*.json ./` y `4. RUN npm install --omit=dev`

- **¿Por qué separamos esto en dos pasos? (La magia del *Layer Caching*):**
  Docker construye las imágenes por capas superpuestas en caché. Si cambiáramos una sola línea de código en `src/server.js` y hubiéramos hecho `COPY . .` al principio, Docker invalidaría toda la caché posterior y tendría que reinstalar todas las librerías con npm cada vez que compilamos.
  Al copiar **solo** el archivo `package.json` primero, Docker detecta si has añadido librerías nuevas o no:
  - Si no has tocado dependencias → Reutiliza la capa ya descargada en **0.1 segundos**.
- El flag `--omit=dev` evita instalar herramientas que solo se usan en desarrollo local, ahorrando espacio en disco.

5. `COPY . .`

- Copia el resto del código de nuestra API (controladores, rutas, modelos) desde la máquina anfitriona hacia `/app` dentro del contenedor.

> **Important**
>
> **No confundir el código en el servidor con el contenedor en ejecución**
>
> Al conectarse por SSH a la instancia EC2, es fácil observar la estructura de directorios (`~/pizzeria-base/backend` , `~/pizzeria-base/frontend` , etc.) y pensar que esas carpetas *son* la instalación o el entorno donde corre el contenedor. **No es así:**
>
> 1. **Función real de las carpetas en el host (EC2):** Son únicamente la copia de trabajo de Git; representan el código fuente o la "materia prima" que Docker necesita para construir las imágenes.
> 2. **Dónde vive el contenedor:** El contenedor se ejecuta como un proceso aislado gestionado por el motor de Docker en memoria RAM, con su propio sistema de archivos independiente (`/app`). De hecho, si una vez arrancado el contenedor se eliminara la carpeta `backend/` del disco de la EC2, la API continuaría funcionando y recibiendo pedidos sin inmutarse, ya que todo su entorno ya fue copiado y empaquetado en la imagen.
> 3. **El ciclo de trabajo real:**
>   - **Paso 1 (`git pull`):** Se descargan las novedades de Git en el disco del host. Esto actualiza los ficheros locales, pero **no** tiene ningún efecto sobre los contenedores que ya están corriendo.
>   - **Paso 2 (`docker compose up -d --build`):** Docker lee esos ficheros actualizados, ejecuta el `Dockerfile` (volviendo a hacer `COPY . .` con el nuevo código), hornea una imagen actualizada y recrea el contenedor sustituyendo al anterior de forma limpia.

6. `EXPOSE 3000`

- Es una instrucción documental de metadatos. Informa a Docker y a otros desarrolladores de que este contenedor escucha tráfico de red por el puerto 3000. No publica el puerto por sí sola hacia el exterior (eso se hace con `-p` en runtime o en Docker Compose).

7. `CMD ["node", "src/server.js"]`

- **La gran diferencia entre `RUN` y `CMD` :**
  - `RUN` se ejecuta **durante la construcción de la imagen** (`docker build`). Escribe cambios permanentes en las capas de la imagen.
  - `CMD` es el comando por defecto que se ejecuta **cuando el contenedor arranca** (`docker run`). En nuestro caso, arranca el servidor web Express de la pizzería.

Capítulo 4: El Kit de Supervivencia: Ciclo de Vida y Terminal

En el día a día administrando la pizzería en AWS EC2, no interactuamos con interfaces gráficas: todo se gestiona desde la terminal. Estos son los **4 comandos indispensables** para diagnosticar y operar el sistema en producción.



1. ¿Quién está vivo y quién ha muerto?: `docker ps` vs `docker ps -a`

- **`docker ps` (Solo los vivos):**
  Muestra únicamente los contenedores que están actualmente encendidos y corriendo (`Up`).

  ```
  docker ps
  ```

  Si tu API se ha caído por un error de sintaxis, **aquí no aparecerá**, lo que puede hacerte creer falsamente que el contenedor no existe.
- **`docker ps -a` (El "tanatorio" de contenedores):**
  El modificador `-a` (*all*) muestra **todos los contenedores**, incluidos los detenidos o los que han fallado (`Exited`).

  ```
  docker ps -a
  ```

  > 💡 **Tip de diagnóstico de códigos de salida:**
  >
  > - `Exited (0)` : Terminó correctamente (su tarea finalizó con éxito).
  > - `Exited (1)` : Error en la aplicación (excepción no controlada en Node.js, fallo de sintaxis, etc.).
  > - `Exited (137)` : Señal `SIGKILL` (128 + 9). Ocurre cuando el kernel de Linux mata el contenedor por falta de memoria RAM (*OOMKilled - Out Of Memory*) o cuando alguien ejecutó `docker kill` / apagado forzoso.

2. Control del Ciclo de Vida: `docker stop` , `docker start` y `docker rm`

- **`docker stop <nombre>` :**
  Envía una señal de apagado ordenado (`SIGTERM`) al proceso para que cierre conexiones con la base de datos limpiamente. Si tras 10 segundos el proceso no ha terminado de cerrarse, Docker envía un `SIGKILL` forzoso.

  ```
  docker stop pizzeria-prod-backend
  ```

- **`docker start <nombre>` :**
  Vuelve a encender un contenedor que estaba apagado sin recrearlo ni perder su estado:

  ```
  docker start pizzeria-prod-backend
  ```

- **`docker rm <nombre>` (Destruir contenedor):**
  Elimina el contenedor del disco. Solo se pueden borrar contenedores que estén previamente detenidos:

  ```
  docker stop pizzeria-prod-backend
  docker rm pizzeria-prod-backend
  ```

  (Para forzar la eliminación de un contenedor bloqueado sin detenerlo antes: `docker rm -f <nombre>`).

3. Diagnóstico en Vivo: `docker logs -f`

En Node.js, todo lo que tu código imprime con `console.log()` o `console.error()` es capturado por Docker.

```
docker logs pizzeria-prod-backend --tail 15
```

Muestra las últimas 15 líneas registradas en el servidor.

Caso de uso real: Auditoría del Webhook de Stripe en vivo

Cuando un cliente paga con tarjeta en Stripe, los servidores de Stripe lanzan una petición HTTP asíncrona hacia nuestro servidor (`POST /api/pagos/webhook`). ¿Cómo sabemos si el evento ha llegado y si la base de datos ha actualizado el pedido?

Añadimos el flag `-f` (*follow* / seguir en directo):

```
docker logs pizzeria-prod-backend -f
```

La terminal se quedará a la escucha. En el instante exacto en que completes el pago en Stripe, verás aparecer en tiempo real el registro programado en nuestro pagosController.js:

```
✅ [Stripe Webhook] Pedido #104 marcado como PAGADO tras confirmación de Stripe.
```

(Y si las claves del webhook estuvieran mal configuradas en el `.env` , verías de inmediato el aviso:
⚠️ `[Stripe Webhook] Error en la firma del webhook: ...`).
Para salir de la visualización en vivo, pulsa `Ctrl + C` .

4. Teletransporte al interior del contenedor: `docker exec -it`

A veces necesitas entrar físicamente "dentro" del contenedor para inspeccionar archivos, probar la red o verificar configuraciones internas.

```
docker exec  -it  pizzeria-prod-web  sh
|            |    |                  └─ Intérprete de comandos (Shell)
|            |    └──────────────────── Contenedor destino
|            └───────────────────────── -i (Interactivo) + -t (Terminal TTY)
└────────────────────────────────────── Ejecutar un comando dentro
```

Caso de uso real: Depuración de Nginx cuando la API no responde

Imagina que un cliente entra a la web y recibe un error `502 Bad Gateway` al pedir pizzas. ¿El fallo está en Nginx o en Node.js?

1. **Entramos dentro del contenedor de Nginx:**

   ```
   docker exec -it pizzeria-prod-web sh
   ```

   > 💡 **Nota sobre el intérprete de comandos en Alpine:**
   > Las imágenes base puras de `nginx:alpine` no traen `bash` instalado por defecto (solo traen `/bin/sh`). Sin embargo, en el Dockerfile del Frontend Web instalamos explícitamente `bash` con `RUN apk add --no-cache openssl bash` para dar soporte a nuestro script de certificados SSL. Por tanto, en nuestro contenedor funcionan tanto `sh` como `bash` . Aun así, acostumbrarse a invocar `sh` es la mejor práctica en el ecosistema Alpine.

2. **Verificamos la sintaxis del archivo de configuración de Nginx:**

```
nginx -t
```

(Si falta un punto y coma o hay una directiva mal escrita, Nginx te dirá la línea exacta).

3. **Probamos si Nginx puede "hablar" con el backend por la red interna:**
   Desde dentro del contenedor de Nginx, lanzamos una petición HTTP directa al nombre DNS del backend en el puerto 3000:

   ```
   wget -qO- http://backend:3000/api/health
   ```

   - Si devuelve `{"status":"UP"...}` → La red de Docker funciona y el backend está sano.
   - Si se queda colgado o da error → El backend está apagado o en bucle de reinicio.
4. **Salir del contenedor:**

   ```
   exit
   ```

Capítulo 5: Persistencia y Redes Aisladas

1. Redes Internas: El DNS Mágico de Docker

Por defecto, cada contenedor que creas nace en su propio ecosistema de red aislado. Si dos contenedores necesitan comunicarse, debemos conectarlos a una misma red virtual (en nuestro caso, una red de tipo *Bridge* llamada `pizzeria-network`).

graph LR subgraph RED ["Red Interna: pizzeria-network (Bridge)"] Nginx["pizzeria-prod-web
(Nginx Proxy)"] Backend["pizzeria-prod-backend
(Node.js :3000)"] end Cliente((Cliente Web)) -->|HTTP :80| Nginx Nginx -->|DNS interno: 'http://backend:3000'| Backend

¿Por qué no usamos direcciones IP? (El DNS interno)

En redes físicas tradicionales, configurarías IPs fijas (ej. `192.168.1.50`). En Docker eso es una mala práctica porque las IPs de los contenedores son dinámicas: cada vez que un contenedor se reinicia o se recompila, Docker le asigna una IP diferente (ej. `172.18.0.3`, `172.18.0.4`).

Para solucionar esto, Docker incluye un **servidor DNS interno integrado**:

- En el archivo de configuración real de nuestro proxy (frontend-web/nginx.conf):

  ```
  location ^~ /api/ {
      proxy_pass http://backend:3000/api/;
  }
  ```

- Nginx no tiene ni idea de qué IP tiene el backend. Simplemente le pide datos a `http://backend:3000`.
- El motor de Docker intercepta la palabra `backend` y la traduce automáticamente y al vuelo a la IP privada que tenga el contenedor en ese instante.

2. Persistencia de Datos: Volúmenes vs. Bind Mounts

El gran problema: Los contenedores son "amnésicos"

El sistema de archivos de un contenedor es efímero. Cuando un contenedor escribe datos (un archivo subido, un log o una tabla), lo guarda en una capa temporal de lectura/escritura. Si ejecutas `docker rm`, **todo lo que había dentro desaparece para siempre**.

Para conservar datos o compartir archivos con el host disponemos de dos mecanismos:

Mecanismo	¿Dónde vive?	Caso de uso ideal
Volumen gestionado (*Named Volume*)	Carpeta interna protegida y gestionada por Docker (`/var/lib/docker/volumes/...`).	Bases de datos locales pesadas. *(En nuestra pizzería delegamos la persistencia en AWS RDS por alta disponibilidad y seguridad).*
Punto de montaje (*Bind Mount*)	Una ruta exacta de tu disco duro físico (ej. `~/pizzeria-base/.htpasswd`).	Archivos de configuración que queremos editar desde fuera sin reconstruir la imagen.

Caso de uso real en la Pizzería: El Bind Mount del `.htpasswd`

En la **Práctica 5 (Hardening y Ciberseguridad)**, para proteger el panel `/dbgate/` con usuario y contraseña sin tener que reconstruir la imagen de Nginx cada vez que cambiamos una clave, añadiréis un *Bind Mount* en docker-compose.app.yml:

```
frontend-web:
  container_name: pizzeria-prod-web
  # ...
  volumes:
    - ./.htpasswd:/etc/nginx/.htpasswd:ro
```

- **`./.htpasswd`:** El archivo que creamos en el Ubuntu de nuestra EC2 con `openssl`.
- **`:/etc/nginx/.htpasswd`:** La ruta dentro del contenedor donde Nginx irá a buscarlo.
- **`:ro` (*Read-Only*): Principio de Mínimo Privilegio.** Nginx solo necesita leer las contraseñas. Si alguien vulnerara el servidor web, no podría alterar el archivo de claves porque el sistema operativo lo bloquea en solo lectura.

Capítulo 6: Docker Compose: El Director de Orquesta

De lo manual a la Infraestructura como Código (IaC)

Hasta ahora hemos visto comandos sueltos (`docker run`, `docker build`). Si tuviéramos que arrancar nuestra pizzería a mano, tendríamos que escribir en la terminal una ristra de comandos interminable conectando puertos, variables y redes:

```
# ¡La forma propensa a errores que nadie usa en producción!
docker run -d --name backend --network pizzeria-network -e DB_HOST=... pizzeria-backend
docker run -d --name web -p 80:80 --network pizzeria-network pizzeria-web
docker run -d --name qr -network pizzeria-network pizzeria-qr
docker run -d --name tunnel --network pizzeria-network cloudflared tunnel run ...
```

Docker Compose resuelve esto aplicando el concepto de **Infraestructura como Código (IaC)**: defines todos los servicios de tu aplicación en un único archivo YAML reproducible y versionable en Git.

Destripando nuestro `docker-compose.app.yml`

A continuación analizamos la estructura real de nuestra capa de aplicación (docker-compose.app.yml):

```yaml
name: pizzeria-app

services:
  # 1. API REST en Node.js (Construida a medida con Dockerfile)
  backend:
    build:
      context: ./backend
      dockerfile: Dockerfile
    container_name: pizzeria-prod-backend
    restart: unless-stopped
    environment:
      NODE_ENV: production
      PORT: ${BACKEND_PORT:-3000}
      DB_HOST: ${DB_HOST:-db}
      STRIPE_SECRET_KEY: ${STRIPE_SECRET_KEY:-}
    networks:
      - pizzeria-network

  # 2. Servidor Web y Proxy Inverso Nginx
  frontend-web:
    build:
      context: ./frontend-web
    container_name: pizzeria-prod-web
    restart: unless-stopped
    ports:
      - "${HTTP_PORT:-80}:80"
    depends_on:
      - backend
    networks:
      - pizzeria-network

  # 3. Túnel Cloudflare Zero Trust (Usa imagen precompilada del registro oficial)
  tunnel:
    image: cloudflare/cloudflared:latest
    container_name: pizzeria-prod-tunnel
    restart: unless-stopped
    command: tunnel run
    environment:
      - TUNNEL_TOKEN=${CLOUDFLARE_TUNNEL_TOKEN}
    depends_on:
      - frontend-web
    networks:
      - pizzeria-network

networks:
  pizzeria-network:
    external: true
```

Elementos clave a entender:

- **`build` vs `image`:**
  - `backend` y `frontend-web` usan `build:` porque son código nuestro y Docker debe cocinar su imagen a partir de su `Dockerfile`.

- `tunnel` usa `image: cloudflare/cloudflared:latest` porque es una herramienta de terceros lista para descargar directamente desde Docker Hub.

- **`restart: unless-stopped` (Resiliencia en Producción):**
  Si la máquina EC2 se reinicia por mantenimiento de AWS o si el proceso del backend se cae por un error inesperado, Docker lo levantará automáticamente. Solo permanecerá apagado si tú lo detienes intencionadamente con `docker stop`.
- **`${VARIABLE:-valor_defecto}`:**
  Inyección limpia desde el archivo `.env`. Si la variable no está en el `.env`, toma el valor por defecto tras los dos puntos.
- **`ports: "80:80"`:**
  Mapea el puerto del Host (EC2) al del contenedor (`Host:Contenedor`). Solo Nginx expone el puerto 80 hacia fuera; el backend queda protegido y accesible únicamente por la red interna.
- **`depends_on` (¡Atención: orden de arranque, no de salud!):**
  Define el orden en el que Docker lanza los contenedores (primero backend, luego web, luego tunnel). **No espera** a que la base de datos o Node.js hayan terminado de compilar o conectar (para eso se usan *Healthchecks* avanzados).
- **`networks: external: true`:**
  Indica que la red `pizzeria-network` ya fue creada previamente (por la capa de base de datos) y se comparte entre ambos entornos.

Comandos Clave del Director de Orquesta

```
# 1. Levantar y compilar toda la infraestructura en segundo plano (-d)
docker compose -f docker-compose.app.yml up -d --build

# 2. Ver el estado de todos los servicios coordinados
docker compose -f docker-compose.app.yml ps

# 3. Recrear un solo servicio tras editar el .env (¡El comando correcto para recargar variables!)
docker compose -f docker-compose.app.yml up -d backend

# 4. Detener y eliminar todos los contenedores y redes de forma limpia
docker compose -f docker-compose.app.yml down
```

Capítulo 7: Chuleta Rápida de Comandos (Cheat Sheet)

Guarda esta tabla como referencia rápida para la terminal de AWS EC2:

Acción deseada	Comando exacto	Para qué sirve
Ver contenedores activos	`docker ps`	Comprueba si tus servicios están encendidos (`Up`).
Ver todos (incluidos caídos)	`docker ps -a`	Localiza qué contenedor ha muerto (`Exited (1)`).
Ver logs en tiempo real	`docker logs -f <nombre>`	Depura peticiones entrantes, errores y eventos en vivo.

Acción deseada	Comando exacto	Para qué sirve
Ver últimas N líneas de log	`docker logs --tail 20 <nombre>`	Consulta el arranque reciente sin saturar la pantalla.
Entrar dentro del contenedor	`docker exec -it <nombre> sh`	Abre una terminal interactiva para probar red o archivos.
Levantar stack con Compose	`docker compose -f <fichero> up -d --build`	Compila cambios de código y arranca todo en segundo plano.
Recargar variables del `.env`	`docker compose -f <fichero> up -d <servicio>`	Recrea el contenedor aplicando las nuevas variables.
Apagar stack completo	`docker compose -f <fichero> down`	Detiene y elimina contenedores y redes de forma ordenada.
Ver consumo de CPU y RAM	`docker stats`	Monitoriza recursos en tiempo real de cada contenedor.

Capítulo 8: Guía Rápida de Troubleshooting (Resolución de Incidencias)

Tres situaciones reales que te ocurrirán en producción y cómo resolverlas en 10 segundos:

1. Error `530 / Error 1033` en Cloudflare nada más arrancar

- **Síntoma:** Abres tu enlace `https://dam-XX.guillermofoix.org` y sale una pantalla de error de Cloudflare (*Argo Tunnel Error*).
- **Causa:** El contenedor del túnel acaba de arrancar y necesita entre 15 y 30 segundos para resolver DNS y registrar sus 4 conexiones Anycast con los servidores de Cloudflare.
- **Solución:** Revisa con `docker logs pizzeria-prod-tunnel -f`. En cuanto veas `Registered tunnel connection...`, recarga la web y cargará de inmediato.

2. Error *"Failed to fetch"* al tramitar pedidos

- **Síntoma:** Pulsas *Confirmar y Enviar Pedido* en la web y salta un aviso rojo `Failed to fetch`.
- **Causa:** El navegador lanzó `POST /api/pedidos` pero el backend o Nginx estaban reiniciándose en ese milisegundo, o el túnel perdió conexión temporal.
- **Solución:** Comprueba que el backend esté arriba con `docker ps`. Si acaba de reiniciar, espera 10 segundos y pulsa `Ctrl + F5` en el navegador.

3. Edito el archivo `.env` pero el servidor no se entera

- **Síntoma:** Has añadido tus claves de Stripe en el `.env` y el backend sigue diciendo `Pasarela inactiva`.
- **Causa:** Has usado `docker restart` o `docker compose restart`. Un reinicio simple **no** lee de nuevo el archivo `.env`.
- **Solución:** Ejecuta siempre:

```
docker compose -f docker-compose.app.yml up -d backend
```

Compose detectará que el archivo de variables ha cambiado, destruirá el contenedor viejo y creará uno nuevo con las claves inyectadas.